🌳 Binary Tree — Complete Quick Refresher Guide

> **Purpose:** A consolidated reference book covering every key Binary Tree concept, pattern, approach and complexity. Sync'd with all latest JavaScript solution files.

📚 Table of Contents

#	Topic	Complexity
1	Introduction & Terminology	—
2	Types of Binary Trees	—
3	Node Structure & Tree Construction	O(N)
4	Recursive Traversals	O(N) / O(H)
5	Iterative Traversals	O(N) / O(N)
6	Level Order Traversal (BFS)	O(N) / O(N)
7	All-in-One Traversal	O(N) / O(N)
8	Height & Depth	O(N) / O(H)
9	Diameter of Binary Tree	$O(N^2) \rightarrow O(N)$
10	Check Balanced	$O(N^2) \rightarrow O(N)$
11	Symmetric / Mirror Tree	O(N) / O(H)
12	Same Tree	O(N) / O(H)
13	Maximum Path Sum	O(N) / O(H)
14	Lowest Common Ancestor (LCA)	O(N) / O(H)
15	Zigzag Level Order	O(N) / O(N)

#	Topic	Complexity
16	Left & Right View	O(N) / O(N)
17	Top View	O(N log N) → O(N)
18	Bottom View	O(N log N) → O(N)
19	Boundary Traversal	O(N) / O(H)
20	Vertical Order Traversal	O(N log N) / O(N)
21	Morris Traversal (O(1) Space)	O(N) / O(1)
22	Serialize & Deserialize	O(N) / O(N)
23	Complexity Summary	—

1. Introduction & Terminology

A **Binary Tree** is a hierarchical data structure where every node has **at most two children** — left and right.

```
      1          ← Root (Level 1, Depth 0)
     / \
    2   3        ← Level 2, Depth 1
   / \
  4   5          ← Level 3, Depth 2  (4 and 5 are Siblings)
```

Term	Definition
Root	Topmost node — no parent
Leaf	Node with no children (`left = right = null`)
Height	# edges on the longest root-to-leaf path. Leaf=0, null=-1
Depth	# edges from root to a given node. Root=0
Level	Depth + 1. Root is at Level 1

Key Formulas

```
Max nodes at level L         = 2^(L-1)
Max nodes in tree of height H = 2^H - 1
Min height for N nodes       = ⌊log₂(N)⌋
Full Binary Tree:             leaves = internal nodes + 1
```

2. Types of Binary Trees

```
Full                 Complete             Perfect              Skewed
0 or 2 children      All full except      All internal=2       One child only
                     last (L→R)           leaves same level    (linked list)
    1                    1                    1                  1
   / \                  / \                  / \                  \
  2   3                2   3                2   3                  2
 / \                  / \ /               / \ / \                  \
4   5                4  5 6             4  5 6  7                   3
```

Balanced: For EVERY node `|height(left) - height(right)| ≤ 1` → guarantees O(log N)

3. Node Structure & Tree Construction

```
class TreeNode {
    constructor(val) { this.val = val; this.left = null; this.right = null; }
}

// Build from level-order array (null = missing node)
function buildTreeFromArray(arr) {
    if (!arr?.length) return null;
    const root = new TreeNode(arr[0]), queue = [root];
    let i = 1;
    while (queue.length > 0 && i < arr.length) {
        const node = queue.shift();
        if (arr[i] != null) { node.left  = new TreeNode(arr[i]); queue.push(node.left);
        if (arr[i] != null) { node.right = new TreeNode(arr[i]); queue.push(node.right)
    }
    return root;
}
```

4. Recursive Traversals

> The only difference between all three is **WHEN the root is visited**.

Order	Pattern	Use Case
Preorder	Root → Left → Right	Copy / Serialize
Inorder	Left → Root → Right	Sorted output in BST
Postorder	Left → Right → Root	Delete tree, evaluate expressions

```
Time: O(N)  |  Space: O(H)
```

```
function preorder(root) {
    const res = [];
    function dfs(node) {
        if (!node) return;
        res.push(node.val); dfs(node.left); dfs(node.right); // ROOT first
    }
    dfs(root); return res;
}

function inorder(root) {
    const res = [];
    function dfs(node) {
        if (!node) return;
        dfs(node.left); res.push(node.val); dfs(node.right); // ROOT middle
    }
    dfs(root); return res;
}

function postorder(root) {
    const res = [];
    function dfs(node) {
        if (!node) return;
        dfs(node.left); dfs(node.right); res.push(node.val); // ROOT last
    }
    dfs(root); return res;
}
```

5. Iterative Traversals

> `stack.pop()` is O(1). Avoids stack-overflow on deep/skewed trees.

5a. Iterative Preorder

```
// Push root → pop → record → push RIGHT then LEFT (LIFO = left processed first)
function preorderIterative(root) {
    if (!root) return [];
    const result = [], stack = [root];
    while (stack.length > 0) {
        const node = stack.pop();
        result.push(node.val);
        if (node.right) stack.push(node.right);
        if (node.left)  stack.push(node.left);
    }
    return result;
}
```

5b. Iterative Inorder

```
// Drill left spine → pop → record → move right → repeat
function inorderIterative(root) {
    const ans = [], stack = [];
    let curr = root;
    while (curr || stack.length > 0) {
        while (curr) { stack.push(curr); curr = curr.left; }
        curr = stack.pop();
        ans.push(curr.val);
        curr = curr.right;
    }
    return ans;
}
```

5c. Iterative Postorder — Two Stack

```
// stack1 drives order, stack2 reverses → Left→Right→Root
function postorderIterative(root) {
    const ans = [];
    if (!root) return ans;
    const stack1 = [root], stack2 = [];
    while (stack1.length > 0) {
        const node = stack1.pop();
        stack2.push(node);
        if (node.left)  stack1.push(node.left);
        if (node.right) stack1.push(node.right);
    }
    while (stack2.length > 0) ans.push(stack2.pop().val);
    return ans;
}
```

5d. Iterative Postorder — One Stack + `prev`

```
// prev tracks last visited node to avoid re-processing right child
function postorderOneStack(root) {
    const ans = [], stack = [];
    let curr = root, prev = null;
    while (curr || stack.length > 0) {
        while (curr) { stack.push(curr); curr = curr.left; }
        const top = stack[stack.length - 1];
        if (top.right && top.right !== prev) {
            curr = top.right;            // Right not yet visited
        } else {
            ans.push(top.val);           // Both subtrees done → visit
            prev = stack.pop();
        }
    }
    return ans;
}
```

> **Why `prev` ?** `top.right === prev` means "we just came back from right subtree → visit root now". Without it we'd push the right child again infinitely.
>
> **Two-Stack Space = O(N)** | **One-Stack Space = O(H)** ← One-Stack is more space-efficient

6. Level Order Traversal (BFS)

```
// Snapshot queue size at each level → exactly those nodes belong to current level
function levelOrder(root) {
    if (!root) return [];
    const result = [], queue = [root];
    while (queue.length > 0) {
        const n = queue.length, level = [];
        for (let i = 0; i < n; i++) {
            const node = queue.shift();
            level.push(node.val);
            if (node.left)  queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
        result.push(level);
    }
    return result;
}
```

```
Time: O(N)  |  Space: O(N) — at most N/2 nodes in queue (last level)
```

7. All-in-One Traversal

> Compute **Pre + In + Post** in a **single DFS pass** using a state counter.

```
state 1 → record Preorder  → re-push with state 2 → push left child
state 2 → record Inorder   → re-push with state 3 → push right child
state 3 → record Postorder → pop

Time: O(3N) ≈ O(N)  |  Space: O(N)
```

```javascript
function allInOneTraversal(root) {
    const pre = [], ino = [], post = [];
    if (!root) return [pre, ino, post];
    const stack = [[root, 1]];
    while (stack.length > 0) {
        const [node, state] = stack.pop();
        if (state === 1) {
            pre.push(node.val);
            stack.push([node, 2]);
            if (node.left)  stack.push([node.left,  1]);
        } else if (state === 2) {
            ino.push(node.val);
            stack.push([node, 3]);
            if (node.right) stack.push([node.right, 1]);
        } else {
            post.push(node.val);   // state 3 → pop, no re-push
        }
    }
    return [pre, ino, post];
}
```

8. Height & Depth

```
// Height (edge count) — null=-1, leaf=0
function heightOfTree(root) {
    if (!root) return -1;
    return 1 + Math.max(heightOfTree(root.left), heightOfTree(root.right));
}

// LeetCode #104 — counts NODES (height + 1)
function maxDepth(root) {
    if (!root) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}

// Alternative — track depth via DFS parameter
function maxDepthDFS(root) {
    let result = 0;
    function dfs(node, depth) {
        if (!node) { result = Math.max(result, depth); return; }
        dfs(node.left, depth + 1); dfs(node.right, depth + 1);
    }
    dfs(root, 0);
    return result;
}
```

> `height` = edges (null=-1), `maxDepth` = nodes (null=0). They differ by 1.

9. Diameter of Binary Tree

> Longest path between **any two nodes** (edge count). May not pass through root.

Brute Force — O(N²)

```
// height() recomputed at every node → O(N) × O(N)
function heightD(node) { return !node ? 0 : 1 + Math.max(heightD(node.left), heightD(no
function diameterBrute(root) {
    if (!root) return 0;
    return Math.max(
        heightD(root.left) + heightD(root.right),   // Through root
        diameterBrute(root.left),
        diameterBrute(root.right)
    );
}
```

Optimal — O(N)

```
// Combined height + diameter in ONE DFS pass via closure
function diameterOfBinaryTree(root) {
    let maxDiameter = 0;
    function dfs(node) {
        if (!node) return 0;
        const left = dfs(node.left), right = dfs(node.right);
        maxDiameter = Math.max(maxDiameter, left + right); // diameter through this nod
        return 1 + Math.max(left, right);                  // height to parent
    }
    dfs(root);
    return maxDiameter;
}
```

10. Check Balanced

Brute Force — O(N²)

```
function isBalancedBrute(root) {
    if (!root) return true;
    const lh = height(root.left), rh = height(root.right);
    return Math.abs(lh - rh) <= 1
        && isBalancedBrute(root.left)
        && isBalancedBrute(root.right);
}
```

Optimal — O(N) with `-1` sentinel

```
// Returns actual height OR -1 if imbalanced → short-circuits immediately
function getHeight(node) {
    if (!node) return 0;
    const left  = getHeight(node.left);
    if (left  === -1) return -1;
    const right = getHeight(node.right);
    if (right === -1) return -1;
    if (Math.abs(left - right) > 1) return -1;
    return 1 + Math.max(left, right);
}
function isBalanced(root) { return getHeight(root) !== -1; }
```

> **Key trick:** `-1` does two jobs — signals "unbalanced" AND short-circuits further recursion.

11. Symmetric / Mirror Tree

> Mirror pairs: `left.left ↔ right.right` (outer), `left.right ↔ right.left` (inner)

Recursive

```
function isSymmetric(root) {
    function isMirror(left, right) {
        if (!left && !right) return true;
        if (!left || !right) return false;
        return left.val === right.val
            && isMirror(left.left,  right.right)
            && isMirror(left.right, right.left);
    }
    return isMirror(root.left, root.right);
}
```

Iterative — Stack (`pop` O(1) > `shift` O(N))

```
function isSymmetricIterative(root) {
    if (!root) return true;
    const stack = [[root.left, root.right]];
    while (stack.length > 0) {
        const [left, right] = stack.pop();
        if (!left && !right) continue;
        if (!left || !right) return false;
        if (left.val !== right.val) return false;
        stack.push([left.left, right.right], [left.right, right.left]);
    }
    return true;
}
```

12. Same Tree

```
function isSameTree(p, q) {
    if (!p && !q) return true;
    if (!p || !q) return false;
    return p.val === q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right
}
```

```
Time: O(N)  |  Space: O(H)
```

13. Maximum Path Sum

> Path = any node sequence connected by edges. Not required to pass through root.

```
function maxPathSum(root) {
    let maxSum = -Infinity;
    function dfs(node) {
        if (!node) return 0;
        const left  = Math.max(0, dfs(node.left));   // Clamp negatives to 0
        const right = Math.max(0, dfs(node.right));
        maxSum = Math.max(maxSum, node.val + left + right); // Full path through node
        return node.val + Math.max(left, right);            // Single branch to parent
    }
    dfs(root); return maxSum;
}
```

> **Rules:** (1) `Math.max(0,...)` ignores negative subtrees. (2) Return single branch — can't split when extending upward.

14. Lowest Common Ancestor (LCA)

```
function lowestCommonAncestor(root, p, q) {
    if (!root) return null;
    if (root.val === p.val || root.val === q.val) return root;
    const left  = lowestCommonAncestor(root.left,  p, q);
    const right = lowestCommonAncestor(root.right, p, q);
    if (left && right) return root;   // p and q on different sides → root is LCA
    return left ?? right;             // Both on same side
}
```

```
Time: O(N)  |  Space: O(H)
```

15. Zigzag Level Order

> BFS with alternating direction. **Index trick** avoids `.reverse()` per level.

```
function zigzagLevelOrder(root) {
    if (!root) return [];
    const result = [], queue = [root];
    let reverse = false;
    while (queue.length > 0) {
        const n = queue.length, level = new Array(n);
        for (let i = 0; i < n; i++) {
            const node = queue.shift();
            level[reverse ? n - 1 - i : i] = node.val;  // Place at correct index direc
            if (node.left)  queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
        result.push(level);
        reverse = !reverse;
    }
    return result;
}
```

16. Left & Right View

```
function rightView(root) {
    if (!root) return [];
    const result = [], queue = [root];
    while (queue.length > 0) {
        const n = queue.length;
        for (let i = 0; i < n; i++) {
            const node = queue.shift();
            if (i === n - 1) result.push(node.val); // Last  = Right View
            // if (i === 0)  result.push(node.val); // First = Left View
            if (node.left)  queue.push(node.left);
            if (node.right) queue.push(node.right);
        }
    }
    return result;
}
```

17. Top View

> FIRST node seen at each HD = top view (BFS guarantees top-to-bottom).

O(N log N) — sort keys

```
function topView(root) {
    const map = new Map(), queue = [[root, 0]];
    while (queue.length > 0) {
        const [curr, hd] = queue.shift();
        if (!map.has(hd)) map.set(hd, curr.val);
        if (curr.left)  queue.push([curr.left,  hd - 1]);
        if (curr.right) queue.push([curr.right, hd + 1]);
    }
    return [...map.keys()].sort((a, b) => a - b).map(hd => map.get(hd));
}
```

O(N) — track minHd/maxHd, linear scan

```
function topViewOptimal(root) {
    const map = new Map(), queue = [[root, 0]];
    let minHd = 0, maxHd = 0;
    while (queue.length > 0) {
        const [curr, hd] = queue.shift();
        if (!map.has(hd)) map.set(hd, curr.val);
        minHd = Math.min(minHd, hd); maxHd = Math.max(maxHd, hd);
        if (curr.left)  queue.push([curr.left,  hd - 1]);
        if (curr.right) queue.push([curr.right, hd + 1]);
    }
    const result = [];
    for (let hd = minHd; hd <= maxHd; hd++) result.push(map.get(hd));
    return result;
}
```

18. Bottom View

> LAST node seen at each HD = bottom view. Always overwrite.

```
Top View    → if (!map.has(hd)) map.set(hd, val)  ← store FIRST
Bottom View → map.set(hd, val)                     ← store LAST (always overwrite)
```

```
function bottomViewOptimal(root) {
    const map = new Map(), queue = [[root, 0]];
    let minHd = 0, maxHd = 0;
    while (queue.length > 0) {
        const [curr, hd] = queue.shift();
        map.set(hd, curr.val);                                  // Always overwrite
        minHd = Math.min(minHd, hd); maxHd = Math.max(maxHd, hd);
        if (curr.left)  queue.push([curr.left,  hd - 1]);
        if (curr.right) queue.push([curr.right, hd + 1]);
    }
    const result = [];
    for (let hd = minHd; hd <= maxHd; hd++) result.push(map.get(hd));
    return result;
}
```

Top vs Bottom — Quick Comparison

```
Tree:       1          hd=0
           / \
          2   3        hd=-1, hd=1
         / \   \
        4  10   10     hd=-2, hd=0, hd=2
            /\
           9  5        hd=-1, hd=1
               \
                6      hd=2
```

hd	Top View	Bottom View	
-2	4	4	
-1	2	9	← 9 is deeper
0	1	9	← 9 is deeper
1	3	5	← 5 is deeper
2	10	6	← 6 is deeper

19. Boundary Traversal

> Anti-clockwise: Left boundary (top-down) → All leaves (L→R) → Right boundary (bottom-up)

```
function boundaryTraversal(root) {
    if (!root) return [];
    const result = [];
    const isLeaf = n => !n.left && !n.right;

    if (!isLeaf(root)) result.push(root.val);   // Root (if not leaf)

    // Left boundary: go left, fallback right, stop before leaf
    let node = root.left;
    while (node) {
        if (!isLeaf(node)) result.push(node.val);
        node = node.left ?? node.right;
    }

    // All leaves via DFS
    function addLeaves(n) {
        if (!n) return;
        if (isLeaf(n)) { result.push(n.val); return; }
        addLeaves(n.left); addLeaves(n.right);
    }
    addLeaves(root);

    // Right boundary: go right, fallback left → collect → reverse
    const rb = [];
    node = root.right;
    while (node) {
        if (!isLeaf(node)) rb.push(node.val);
        node = node.right ?? node.left;
    }
    result.push(...rb.reverse());
    return result;
}
```

20. Vertical Order Traversal

> Group by `(col, row)`. Root=(0,0). Left=(col-1,row+1). Right=(col+1,row+1). Same (col,row) → sort by value.

```javascript
function verticalOrderTraversal(root) {
    if (!root) return [];
    const map = new Map();
    const queue = [[root, 0]];
    let level = 0;
    while (queue.length > 0) {
        const n = queue.length;
        for (let i = 0; i < n; i++) {
            const [curr, col] = queue.shift();
            (map.get(col) ?? map.set(col, []).get(col)).push([level, curr.val]);
            if (curr.left)  queue.push([curr.left,  col - 1]);
            if (curr.right) queue.push([curr.right, col + 1]);
        }
        level++;
    }
    return [...map.keys()]
        .sort((a, b) => a - b)
        .map(col =>
            map.get(col)
                .sort((a, b) => a[0] - b[0] || a[1] - b[1])
                .map(([, val]) => val)
        );
}
```

> We track `level` because nodes at same `(col, row)` must be sorted by value — row is needed to group them.

21. Morris Traversal (O(1) Space)

> Temporarily threads tree links to avoid stack/recursion. Fully restores the tree.

```
1st visit curr: predecessor.right = null  → create thread → move left
2nd visit curr: predecessor.right = curr  → remove thread → visit → move right

Time: O(N) — each node visited at most twice  |  Space: O(1)
```

```
function morrisInorder(root) {
    const result = [];
    let curr = root;
    while (curr) {
        if (curr.left) {
            let pred = curr.left;
            while (pred.right && pred.right !== curr) pred = pred.right;
            if (pred.right === curr) {   // 2nd visit
                pred.right = null; result.push(curr.val); curr = curr.right;
            } else {                     // 1st visit
                pred.right = curr; curr = curr.left;
            }
        } else {
            result.push(curr.val); curr = curr.right;
        }
    }
    return result;
}
```

```
Morris Dry Run on:  1
                   / \
                  2   3
                 / \
                4   5

curr=1 → pred=5, thread 5→1, go left to 2
curr=2 → pred=4, thread 4→2, go left to 4
curr=4 → no left, visit 4, go right (→2 via thread)
curr=2 → pred=4, 4.right==2, remove, visit 2, go to 5
curr=5 → no left, visit 5, go right (→1 via thread)
curr=1 → pred=5, 5.right==1, remove, visit 1, go to 3
curr=3 → no left, visit 3

Result: [4, 2, 5, 1, 3] ✅
```

22. Serialize & Deserialize

```
function serialize(root) {
    if (!root) return "null";
    const result = [], queue = [root];
    while (queue.length > 0) {
        const node = queue.shift();
        if (!node) { result.push("null"); continue; }
        result.push(String(node.val));
        queue.push(node.left, node.right);
    }
    return result.join(",");
}

function deserialize(data) {
    if (data === "null") return null;
    const tokens = data.split(",");
    const root = new TreeNode(Number(tokens[0])), queue = [root];
    let i = 1;
    while (queue.length > 0 && i < tokens.length) {
        const node = queue.shift();
        if (tokens[i] !== "null") { node.left  = new TreeNode(Number(tokens[i])); queue
        if (tokens[i] !== "null") { node.right = new TreeNode(Number(tokens[i])); queue
    }
    return root;
}
```

23. Complexity Summary

Operation	Time	Space	Notes
Recursive Pre/In/Post	O(N)	O(H)	
Iterative Preorder	O(N)	O(N)	
Iterative Inorder	O(N)	O(N)	
Iterative Postorder — Two Stack	O(N)	O(N)	

Operation	Time	Space	Notes
Iterative Postorder — One Stack + prev	O(N)	O(H)	More space-efficient
Level Order (BFS)	O(N)	O(N)	max N/2 in queue
All-in-One Traversal	O(N)	O(N)	single pass for all 3
Height / MaxDepth	O(N)	O(H)	
MaxDepth DFS with depth param	O(N)	O(H)	alternative approach
Diameter Brute	$O(N^2)$	O(H)	height recomputed per node
Diameter Optimal	O(N)	O(H)	combined DFS pass
Balanced Brute	$O(N^2)$	O(H)	height recomputed per node
Balanced Optimal (-1 sentinel)	O(N)	O(H)	short-circuits on imbalance
Symmetric — Recursive	O(N)	O(H)	
Symmetric — Iterative Stack	O(N)	O(H)	pop O(1) > shift O(N)
Same Tree	O(N)	O(H)	
Maximum Path Sum	O(N)	O(H)	clamp negatives to 0
Lowest Common Ancestor	O(N)	O(H)	
Zigzag Level Order	O(N)	O(N)	index trick avoids .reverse()
Left / Right View	O(N)	O(N)	first/last per BFS level
Top/Bottom View (sort keys)	O(N log N)	O(N)	
Top/Bottom View (minHd/maxHd)	O(N)	O(N)	no sort needed
Boundary Traversal	O(N)	O(H)	3-part: L + leaves + R reversed
Vertical Order Traversal	O(N log N)	O(N)	sort by col, row, val
Morris Inorder / Preorder	O(N)	**O(1)**	no stack, no recursion

Operation	Time	Space	Notes
Serialize / Deserialize	O(N)	O(N)	BFS with null markers

```
H = height:  Balanced → O(log N)  |  Skewed → O(N)
```

🔑 Key Patterns at a Glance

Pattern	Problems
DFS + closure variable	Diameter, Max Path Sum — track global max while returning local height
-1 sentinel	Balanced check — single value signals imbalance AND short-circuits
BFS level snapshot	Level Order, Zigzag, Views — `const n = queue.length` before inner loop
HD (horizontal distance)	Top/Bottom/Vertical Order — col tracking via BFS queue
minHd/maxHd tracking	Top/Bottom View O(N) — replace key sort with linear scan
`prev` pointer	One-stack Postorder — avoids re-pushing already-visited right child
Index placement trick	Zigzag — `level[reverse ? n-1-i : i]` avoids `.reverse()`
Mirror pair comparison	Symmetric — always compare outer+inner pairs, not same-side
Thread + restore	Morris — O(1) space by wiring predecessor.right temporarily
Two non-null sides = LCA	LCA — if both left/right return non-null, current node is the answer

Sync'd with: `1. Traversals/javascript/` and `2. Medium Problems/javascript/` — Strivers A2Z DSA Sheet